

Catering and Inventory Management System

Software Requirements Specification

Version 1.0

Revision History

Date	Version	Description	Author
April 2026	1.0	Initial draft	Betopia Product Team

Table of Contents

- 1. Introduction**
 - 1.1. Purpose
 - 1.2. Definitions, Acronyms and Abbreviations
 - 1.3. Overview
- 2. Overall System Description**
- 3. Scope**
 - 3.1. In Scope
 - 3.2. Out of Scope
- 4. System Architecture Overview**
 - 4.1. Frontend
 - 4.2. Backend
 - 4.3. Database
 - 4.4. Infrastructure
- 5. Functional Requirements**
 - 5.1. Authentication (FR-01)
 - 5.2. User Management (FR-02)
 - 5.3. Menu Management (FR-03)
 - 5.4. Automatic Menu Availability Control (FR-04)
 - 5.5. On-Demand Ordering (FR-05)
 - 5.6. Order Queue & Status Management (FR-06)
 - 5.7. Cash Transaction Recording (FR-07)
 - 5.8. Inventory Tracking (FR-08)
 - 5.9. Admin Dashboard & Reporting (FR-09)
 - 5.10. Access Control (FR-10)
- 6. Non-Functional Requirements**
 - 6.1. Performance (NFR-01)

- 6.2. Reliability (NFR-02)
- 6.3. Security (NFR-03)
- 6.4. Usability (NFR-04)
- 6.5. Scalability (NFR-05)
- 6.6. Data Integrity (NFR-06)
- 7. **Assumptions & Dependencies**
 - 7.1. Assumptions
 - 7.2. Dependencies
- 8. **Risk Matrix**

Software Requirements Specification

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete software requirements for KitchenFlow, a web-based catering and inventory management platform developed by Betopia Limited. It describes the system's functional behavior, non-functional standards, architecture, and technical constraints to a level of detail sufficient for the development team to design, build, and test the platform. This document serves as the primary technical reference throughout all phases of development.

1.2 Definitions, Acronyms and Abbreviations

Term	Definition
SRS	Software Requirements Specification
BRD	Business Requirements Document
FR	Functional Requirement
NFR	Non-Functional Requirement
API	Application Programming Interface
REST	Representational State Transfer

RBAC	Role-Based Access Control
Admin	The organizational administrator managing users, menu, and inventory
Kitchen Staff	Staff responsible for fulfilling orders and managing stock
User	An employee browsing the menu and placing orders

1.3 Overview

This document is organized into eight sections. The introduction and overall system description provide context and background. The scope defines the boundaries of the system. The architecture overview describes the technical stack and component structure. Functional and non-functional requirements detail what the system must do and how it must perform. Assumptions and dependencies outline the external factors the system relies on. The risk matrix identifies technical risks and mitigations.

2. Overall Description

KitchenFlow is a role-based, web-based catering and inventory management platform designed for organizations that operate an internal catering or canteen service. It digitizes the full catering workflow, from meal ordering and cash transaction recording to real-time inventory tracking, within a single unified system, eliminating the need for any parallel paper-based process.

The platform serves three distinct user roles, each operating within clearly defined boundaries:

- **Admin:** Manages users, menu items, stock levels, and has full visibility into daily operations through a reporting dashboard.
- **Kitchen Staff:** Receives and processes incoming orders through a live order queue, manages item availability, and updates order statuses in real time.
- **User:** Browses the available menu and places on-demand orders during active service periods.

The system operates entirely through a web browser, requiring no native mobile application or hardware installation. It is hosted and maintained by Betopia Limited and delivered to client organizations as a subscription-based service. Each organization operates within its own isolated instance of the platform.

The platform is designed for use during active catering service hours, where real-time responsiveness between Users placing orders and Kitchen Staff fulfilling them is critical. Outside of service hours, Admins use the system for stock management, menu configuration, user administration, and reviewing operational records.

3. Scope

3.1 In Scope

KitchenFlow Phase 1 covers the following:

1. User registration, login, and role-based access control for Admins, Kitchen Staff, and Users.
2. Menu management including item creation, pricing, category assignment, and availability control.
3. On-demand meal ordering with a real-time order queue for Kitchen Staff.
4. *Order status management through the flow: Pending → Preparing → Ready → Served / Canceled.*
5. Cash transaction recording per order, including amount tendered and change returned.
6. Portion-based inventory tracking with low-stock alerts and automatic item disabling at zero stock.
7. Admin dashboard with daily order summaries, revenue records, and top ordered items, filterable by date.

3.2 Out of Scope

The following are explicitly excluded from Phase 1 and will not be covered by this document:

1. Digital wallet, prepaid balance, or any online payment method.
2. Multi-branch or multi-location catering operations.
3. Third-party payment gateway or external banking integration.
4. Native mobile application for iOS or Android.
5. Integration with external systems such as HR, ERP, or accounting tools.
6. Advanced analytics, forecasting, or AI-driven insights.

4. System Architecture Overview

KitchenFlow is built as a decoupled web application consisting of three core layers: a frontend

client, a backend API server, and a relational database. All components are containerized and

orchestrated via Docker Compose.

4.1 Frontend

Built with Next.js 14 using the App Router, TypeScript, and Tailwind CSS. Serves the user interface for all three roles: Admin, Kitchen Staff, and User. Server Components are used by default, with Client Components only where interactivity requires it. The frontend communicates with the backend exclusively through REST API calls via NEXT_PUBLIC_API_URL.

4.2 Backend

Built with Laravel 11 on PHP 8.2. Exposes a REST API only, no server-rendered views. Handles all business logic including role-based access control, order processing, inventory management, and cash transaction recording. Follows a Service Layer pattern with Resource Controllers, Form Requests, and API Resources.

4.3 Database

PostgreSQL 16 is used as the primary relational database. All data including users, roles, menu items, orders, transactions, and stock levels, is persisted here. All database interaction is handled through Laravel's Eloquent ORM. No raw SQL is used. A tenants table serves as the highest-level entity, scoping all organizational data. Subscription status is enforced at the middleware layer on every API request before any role-based check.

4.4 Infrastructure

Nginx (Alpine) acts as the web server and reverse proxy, routing incoming traffic to the appropriate service. Node 20 LTS and Composer 2 are used as part of the build environment. The full application stack is containerized using Docker Compose for consistency across development and production environments.

5. Functional Requirements

5.1 Authentication (FR-01)

1. **Registration:** Admins must be able to register new user accounts on behalf of employees. Self-registration must not be available to general Users.
2. **Login:** Users must be able to log into the system using their credentials (email and password).
3. **Logout:** Users must be able to securely log out of the system from any page.
4. **Password Reset:** Users must be able to request a password reset and set a new password through a secure reset flow.

5.2 User Management (FR-02)

1. **Create User:** Admins must be able to create new user accounts, specifying name, email, password, and role.

2. **Edit User:** Admins must be able to update any user's name, email, or assigned role.
3. **Deactivate User:** Admins must be able to deactivate a user account, immediately revoking their ability to log in or access the system.
4. **Reactivate User:** Admins must be able to reactivate a previously deactivated account, restoring full access based on their assigned role.
5. **View All Users:** Admins must be able to view a list of all registered users, including their role and current account status.

5.3 Menu Management (FR-03)

1. **Create Menu Item:** Admins must be able to create a new menu item with a name, price, category, and availability status.
2. **Edit Menu Item:** Admins must be able to update any detail of an existing menu item including name, price, and category.
3. **Remove Menu Item:** Admins must be able to permanently remove a menu item from the system.
4. **Manual Availability Toggle:** Kitchen Staff must be able to manually mark any menu item as unavailable at any time during service, independent of its stock level.
5. **View Menu Items:** Admins and Kitchen Staff must be able to view all menu items regardless of availability status. Users must only see currently available items.

5.4 Automatic Menu Availability Control (FR-04)

1. **Auto-Disable at Zero Stock:** The system must automatically disable a menu item from the ordering menu the moment its stock quantity reaches zero.
2. **No Auto Re-enable:** The system must not automatically re-enable a disabled item. Re-enabling must always require a manual action.
3. **Manual Re-enable After Restock:** Kitchen Staff must explicitly confirm a restock has been completed before the system allows a disabled item to be made available again.

5.5 On-Demand Ordering (FR-05)

1. **Browse Menu:** Users must be able to browse all currently available menu items, organized by category.
2. **Place Order:** Users must be able to select one or more menu items and submit an order in real time.
3. **Unavailable Item Restriction:** Items that are disabled either manually or due to zero stock, must not be visible or selectable by Users during ordering.
4. **Order Confirmation:** Upon successful order placement, the User must receive an on-screen confirmation with their order details.

5.6 Order Queue & Status Management (FR-06)

1. **Live Order Queue:** Kitchen Staff must have access to a real-time order queue displaying all incoming orders as they are placed.
2. **Status Update:** *Kitchen Staff must be able to update each order's status through the defined flow: Pending → Preparing → Ready → Served, or Pending → Canceled.*
3. **Real-Time Reflection:** All status updates must be reflected immediately across the system without requiring a page refresh.
4. **Cancel Order:** Kitchen Staff must be able to cancel a Pending order, with the cancellation recorded against the order record.
5. **Order Details View:** Kitchen Staff must be able to view the full details of each order in the queue, including items ordered and the time the order was placed.

5.7 Cash Transaction Recording (FR-07)

1. **Record Tendered Amount:** At the point of serving, the system must allow Kitchen Staff to record the cash amount tendered by the User.
2. **Automatic Change Calculation:** The system must automatically calculate and display the change to be returned based on the order total and the amount tendered.
3. **Transaction Persistence:** The full transaction record including order total, tendered amount, change returned, and timestamp must be saved against the order and remain permanently accessible.
4. **No Negative Change:** The system must prevent recording a tendered amount that is less than the order total.

5.8 Inventory Tracking (FR-08)

1. **Set Stock Quantity:** Admins must be able to set an initial stock quantity for each menu item, defined in portions.
2. **Update Stock Quantity:** Admins must be able to manually update the stock quantity of any item at any time, for example after a restock.
3. **Real-Time Decrementation:** The system must automatically decrement the stock quantity of each ordered item in real time as orders are placed.
4. **Zero Floor:** Stock levels must never be decremented below zero under any circumstance.
5. **Stock Visibility:** Admins and Kitchen Staff must be able to view current stock levels for all menu items at any time.

5.9 Admin Dashboard & Reporting (FR-09)

1. **Daily Order Summary:** The dashboard must display a summary of all orders placed on the current day, including total order count and status breakdown.
2. **Revenue Summary:** The dashboard must display the total cash revenue collected for the current day.

3. **Top Ordered Items:** The dashboard must display the most frequently ordered menu items for the selected date range.
4. **Date Filtering:** All order and transaction records on the dashboard must be filterable by a specific date or date range.
5. **Real-Time Data:** The dashboard must reflect current data at all times without requiring a manual page refresh.

5.10 Access Control (FR-10)

1. **Route Protection:** All routes must be protected server-side, ensuring unauthenticated users cannot access any part of the system.
2. **Role-Based Access:** Each role (Admin, Kitchen Staff, and User) must only be able to access pages and perform actions permitted within their defined scope.
3. **Cross-Role Prevention:** Any attempt by a user to access a resource or perform an action outside their role must be blocked and returned with an appropriate error response.
4. **Session Management:** User sessions must be invalidated immediately upon logout, preventing any further authenticated access without re-login.

6. Non-Functional Requirements

6.1 Performance (NFR-01)

1. **Page Load Time:** All core pages must load within 3 seconds under normal operating conditions.
2. **Order Processing Speed:** Order placement must be processed and reflected in the Kitchen Staff queue within 3 seconds of submission.
3. **Dashboard Load Time:** The Admin dashboard must load and display current data within 3 seconds under normal operating conditions.

6.2 Reliability (NFR-02)

1. **Uptime:** The system must remain available during operational hours with minimal downtime, ensuring no interruption to active catering service.
2. **Error Recovery:** The system must handle unexpected errors gracefully without data loss, returning the user to a stable state.
3. **Data Persistence:** No order, transaction, or stock record must be lost in the event of a system error or unexpected shutdown.

6.3 Security (NFR-03)

1. **Password Encryption:** All user passwords must be encrypted at rest using a recognized hashing algorithm.
2. **Server-Side Enforcement:** Access control for all roles must be enforced server-side on every request, not only at the frontend level.

3. Session Security: User sessions must be securely managed and immediately invalidated upon logout.

6.4 Usability (NFR-04)

1. No Training Required: The interface must be simple enough for Kitchen Staff and Users to operate independently without formal training or a user manual.
2. Role-Appropriate UI: Each role must only see the interface elements and actions relevant to their scope, avoiding unnecessary complexity.
3. Operational Efficiency: Core actions like placing an order, updating an order status, and recording a cash transaction must each be completable in under 3 steps.

6.5 Scalability (NFR-05)

1. User Growth: The system must support a growing number of registered users and daily orders without requiring architectural changes.
2. Concurrent Orders: The system must handle multiple simultaneous orders during peak service periods without performance degradation.

6.6 Data Integrity (NFR-06)

1. Record Accuracy: Order records, cash transactions, and stock levels must remain accurate and consistent at all times.
2. Stock Consistency: Stock decrements must be atomic. A failed order must never result in a stock level change.
3. Transaction Completeness: Every completed order must have a fully recorded cash transaction with no missing fields.

7. Assumptions & Dependencies

7.1 Assumptions

1. Users will access the platform through a modern web browser on a device with a stable internet connection during service hours.
2. Kitchen Staff will actively monitor the order queue and update order statuses in real time throughout the service period.
3. Admins will configure menu items, stock quantities, and user accounts accurately before each service period begins.
4. Only one organization operates per instance of the platform. No multi-branch or multi-tenant setup is assumed.
5. Users will have been issued their login credentials by an Admin prior to using the system. Self-registration is not available.
6. Stock quantities entered by Admins are assumed to reflect actual physical stock available in the kitchen.

7.2 Dependencies

1. Laravel 11 (PHP 8.2)

The entire backend API is dependent on this framework and runtime version.

2. Next.js 14 (Node 20 LTS)

The frontend is dependent on this framework and Node version for build and runtime.

3. PostgreSQL 16

All application data is dependent on this database engine.

4. Nginx (Alpine)

The system depends on Nginx as the web server and reverse proxy for routing traffic between services.

5. Docker Compose

Local development and production deployment are dependent on Docker Compose for container orchestration.

6. Composer 2

The backend build process is dependent on Composer for PHP package management.

7. SMTP / Mail Service

The password reset flow is dependent on an external mail service for delivering reset emails.

8. Risk Matrix

#	Risk	Likelihood	Impact	Mitigation
R-01	Race condition on simultaneous orders Two users ordering the same low-stock item simultaneously may cause stock to decrement below zero.	Medium	High	Wrap stock decrement operations in atomic database transactions. Validate stock availability at the point of order processing on the server side, not only at the UI level.
R-02	Role escalation attempt A User or Kitchen Staff attempts to access Admin-only routes or perform actions outside their permitted scope.	Medium	High	Enforce all role-based access checks server-side via middleware on every request. Never rely on frontend-only route guards as the sole protection.

R-03	Data loss on server error An unexpected system crash or error during an active order causes incomplete records like missing transactions, incorrect stock levels, or orphaned orders.	Low	Critical	Use PostgreSQL transactions for all multi-step operations. Ensure any failure rolls back the entire operation, leaving data in a consistent state.
R-04	Performance degradation during peak service A surge in simultaneous orders during peak canteen hours causes slow response times or request failures.	Medium	High	Optimize Eloquent queries and avoid N+1 problems. Implement API response caching where appropriate. Monitor query performance during load testing before go-live.
R-05	Incorrect cash transaction recording Kitchen Staff records an incorrect tendered amount, resulting in wrong change calculation saved permanently against the order.	Medium	Medium	Validate tendered amount server-side to ensure it is never less than the order total. Consider making transaction records editable by Admin only within a defined correction window.
R-06	Stale order queue for Kitchen Staff The Kitchen Staff order queue does not reflect newly placed orders in real time, causing missed or delayed order fulfillment.	Low	High	Implement polling or a real-time update mechanism on the order queue. Ensure the frontend refreshes queue data at a defined interval without requiring manual page reload.